

Algoritmi-Modulo algoritmi per bioinformatica

Per cominciare-Capitolo 2 CLRS

Roberto Posenato

versione 2026-3, 16/03/2026

- 1 Per cominciare
 - Ordinamento per inserimento (INSERTIONSORT)
- 2 Analisi degli algoritmi
 - Modello di computer
 - Dimensione d'input e tempo di esecuzione
 - Analisi del caso peggiore e del caso medio
 - Tasso di crescita, ordine di grandezza
- 3 Progettare gli algoritmi

*“Tell me and I forget, teach me and I remember,
involve me and I learn.”*

Benjamin Franklin

*“A student who achieves knowledge through free investigation
and spontaneous effort will be able to retain that knowledge
and will have acquired a methodology that can serve for a li-
fetime.”*

Jean Piaget

Prendere maggiore familiarità con i concetti fondamentali della progettazione e dell'analisi degli algoritmi:

- Si studiano due algoritmi di ordinamento costruiti con due tecniche distinte.
- Si analizza la loro complessità computazionale.
- Si introduce il concetto di *ordine di grandezza* per la complessità computazionale.

Ordinamento per inserimento (INSERTIONSORT)

PROBLEMA ORDINAMENTO

INPUT: Sequenza finita di elementi $\langle a_1, \dots, a_n \rangle$ confrontabili secondo la relazione $\leq$.

OUTPUT: Una permutazione $\langle a'_1, \dots, a'_n \rangle$ dell'input tale che
$$a'_1 \leq a'_2 \leq \dots \leq a'_n.$$

- Assumiamo che l'input sia rappresentato da un array (gli elementi sono accessibili in modo diretto).
- Un primo algoritmo che risolve il problema (algoritmo di ordinamento) è l'algoritmo per Inserimento (INSERTIONSORT).

Ordinamento per inserimento (INSERTIONSORT)

Algoritmo: INSERTIONSORT

Input: $A = [a_1, \dots, a_n]$, array di elementi confrontabili con $\leq$

Output: A ordinato rispetto a $\leq$

```
1: for  $i = 2$  to  $n$  do  
2:    $key = A[i]$   
   // Inserisce  $A[i]$  nella parte  
   // ordinata  $a_1, \dots, a_{i-1}$   
3:    $j = i - 1$   
4:   while  $j > 0$  and  $A[j] > key$  do  
5:      $A[j + 1] = A[j]$   
6:      $j = j - 1$   
7:    $A[j + 1] = key$   
8: return  $A$ 
```

Simulazione

Ordina gli elementi **sul posto**.

Ordinamento per inserimento (INSERTIONSORT)

Esempio 1

Ogni blocco rappresenta un ciclo for (0 o più cicli while)

1	2	3	4	5		<i>Commento</i>
5	2	4	6	1		Seq. ordinata: 5 Scambio tra 5 e 2
j	i					
2	5	4	6	1		Seq. ordinata: 2,5 Scambio tra 5 e 4
←	j	i				
2	4	5	6	1		Seq. ordinata: 2,4,5 Nessun scambio
	←	j	i			
2	4	5	6	1		Seq. ordinata: 2,4,5,6 1 scende fino alla I posizione
		←	j	i		
1	2	4	5	6	i	Sequenza ordinata

Simulatore: <https://liveexample.pearsoncmg.com/dsanimation13ejava/InsertionSortNew.html>

Ordinamento per inserimento (INSERTIONSORT)

Invariante di ciclo

Invariante di ciclo

In un algoritmo, un'*invariante di ciclo* è un'*asserzione* (proprietà booleana) sempre vera all'inizio di ogni interazione del ciclo.

Esempio 2

Nell'INSERTIONSORT, *“all'inizio del ciclo **for**, il sottoarray $A[1 : i - 1]$ è ordinato e formato dagli stessi elementi che erano in $A[1 : i - 1]$.”*

- Un invariante di ciclo può essere utile per dimostrare la correttezza di un algoritmo.
- La validità di un invariante di ciclo si dimostra solitamente per “induzione”.
 - ① **Inizializzazione:** è vera prima della prima iterazione;
 - ② **Conservazione:** rimane vera prima di ogni successiva iterazione;

Ordinamento per inserimento (INSERTIONSORT)

Invariante di ciclo

Esempio 3

Nell'INSERTIONSORT, *“all’inizio del ciclo **for**, il sottoarray $A[1 : i - 1]$ è ordinato e formato dagli stessi elementi che erano in $A[1 : i - 1]$.”*

- ➊ **Inizializzazione:** $i = 2$, $A[1 : i - 1] = A[1 : 1]$ è ordinato perché è un solo elemento. **Vero**
- ➋ **Conservazione:** *informalmente*, $\text{key} = A[i]$. Il ciclo **while** sposta $A[i - 1], A[i - 2], \dots$ a destra di una posizione finché non trova una posizione corretta per **key**, che viene quindi inserita. Quindi il sottoarray $A[1 : i]$ diventa ordinato ed è formato dagli elementi del sottoarray $A[1 : i]$ (senza aggiungerne altri). All’inizio della successiva iterazione, i è aumentato di 1, quindi il sottoarray diventa $A[1 : i - 1]$. **Vero**

Ordinamento per inserimento (INSERTIONSORT)

Invariante di ciclo

Una volta dimostrata la validità di un invariante di ciclo, la si valuta dopo la fine del ciclo (**Conclusione**).

- ③ **Conclusione:** la proprietà dell'invariante e la condizione di terminazione del ciclo forniscono una proprietà utile a dimostrare che l'algoritmo è corretto.

Esempio 4

Nell'INSERTIONSORT, *“all'inizio del ciclo **for**, il sottoarray $A[1 : i - 1]$ è ordinato e formato dagli stessi elementi che erano in $A[1 : i - 1]$.”*

- ③ **Conclusione:** il ciclo termina quando $i = n + 1$. Per l'invariante di ciclo, $A[1 : i - 1] = A[1 : n]$ è ordinato e formato dagli stessi elementi.

l'intero array è ordinato.



l'algoritmo è corretto.

Analisi degli algoritmi

Modello di computer

- Per analizzare la complessità algoritmica, si deve avere un modello di computazione di riferimento.
- In questo corso consideriamo un generico modello di calcolo a **un** processore con memoria ad accesso casuale: **random-access machine (RAM)**.
- Caratteristiche del modello RAM:
 - numeri interi e a virgola mobile.
 - dimensione limitata per ciascuna **parola** (*word*) di memoria (64 bit per esempio).
 - istruzioni elementari che coinvolgono interi/virgola mobile: lettura, scrittura, somma, sottrazione, moltiplicazione, divisione, confronto.
 - istruzioni di **if** e di **salto** (*jump*).
 - istruzioni di chiamata a procedura.
- Ogni istruzione *richiede* un tempo **costante** per essere eseguita.
⇒ *Semplificazione con qualche **zona grigia**, che eviteremo in questo insegnamento.*

Analisi degli algoritmi

Dimensione d'input

- In generale, il tempo richiesto da un algoritmo cresce con la dimensione (lunghezza, taglia) dell'input.
- La **dimensione dell'input** è una misura della quantità di bit/word necessari per rappresentare l'input.
- Non è importante dare la misura esatta ma un indicatore "corretto":
 - nel problema di ordinamento o ricerca in un array, è il numero di elementi.
 - nel problema di Fibonacci, è il numero di bit richiesti per rappresentare l'intero in input (e non il valore dell'input come abbiamo fatto nella prima lezione)
 - ...
- Un indicatore è **corretto** se la misura reale della dimensione d'input è in relazione polinomiale con l'indicatore scelto.

Analisi degli algoritmi

Dimensione d'input

Esempio 5

- Si supponga che l'input I di un algoritmo sia una matrice di interi $n \times n$.
- Se ogni intero richiede una parola di k bit per essere rappresentato, una riga della matrice richiede nk bit + una parola di k bit per indicare la fine della riga (questa ultima parola deve avere un formato diverso da qualsiasi intero rappresentabile).
- L'intera matrice richiede quindi $n(k(n + 1))$ bit. Si scrive $|I| = n(k(n + 1))$ bit.
- Solitamente però si indica che la dimensione dell'input è $|I| = n$, numero di righe.
- Tale indicatore è corretto perché, nel modello RAM in cui k è costante, la dimensione reale $n(k(n + 1))$ è un polinomio in n .

Esempio 6

- Si supponga che l'input I di un algoritmo sia un valore intero n .
- Si deve valutare la dimensione $|I|$ per valori crescenti di n .
- Il valore n è rappresentabile in binario con $\leq \lfloor \lg n \rfloor + 1$ bit.
- Quindi la dimensione reale dell'input è $|I| = \lfloor \lg n \rfloor + 1$.
- **Non si può dire che la dimensione dell'input è il valore n perché $|I|$ è in relazione esponenziale con n . Infatti $2^{|I|} \approx n$.**
- Questo errore capita spesso.

Analisi degli algoritmi

Tempo di esecuzione

Definizione 1 (Tempo di esecuzione)

Il **tempo di esecuzione** di un algoritmo per un **dato** input di dimensione n è il *numero totale di operazioni elementari* eseguite (*passi*) nel modello RAM.

Analisi degli algoritmi

Analisi INSERTIONSORT

INSERTION-SORT(A, n)	<i>cost</i>	<i>times</i>
1 for $i = 2$ to n	c_1	n
2 $key = A[i]$	c_2	$n - 1$
3 <i>// Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$.</i>	0	$n - 1$
4 $j = i - 1$	c_4	$n - 1$
5 while $j > 0$ and $A[j] > key$	c_5	$\sum_{i=2}^n t_i$
6 $A[j + 1] = A[j]$	c_6	$\sum_{i=2}^n (t_i - 1)$
7 $j = j - 1$	c_7	$\sum_{i=2}^n (t_i - 1)$
8 $A[j + 1] = key$	c_8	$n - 1$

c_k = costo esecuzione istruzione alla riga k .

$t_i - 1$ = # cicli **while** nel i -esimo ciclo **for**.

$$T(n) = c_1 n + c_2(n - 1) + c_4(n - 1) +$$

$$c_5 \sum_{i=2}^n t_i + c_6 \sum_{i=2}^n (t_i - 1) + c_7 \sum_{i=2}^n (t_i - 1) + c_8(n - 1)$$

Analisi degli algoritmi

Analisi INSERTIONSORT: caso migliore

- Anche fissando n , il tempo di esecuzione di INSERTIONSORT dipende dalla disposizione dei valori in input (*qualità* dell'input)
- **Caso migliore:** input è già ordinato.

Il corpo del ciclo **while** non viene mai eseguito:

$$\begin{aligned} T(n) &= c_1 n + c_2(n - 1) + c_4(n - 1) + c_5(n - 1) + c_8(n - 1) \\ &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) \Leftarrow \text{f. lineare} \end{aligned}$$

Analisi degli algoritmi

Analisi INSERTIONSORT: caso peggiore

- Anche fissando n , il tempo di esecuzione di INSERTIONSORT dipende dalla disposizione dei valori in input (*qualità* dell'input)
- **Caso peggiore:** l'input è ordinato in ordine inverso. Ogni $A[i]$ deve essere confrontato con tutti gli elementi in $A[1 : i - 1]$. Quindi $t_i = i$.

$$\begin{aligned} T(n) &= c_1 n + c_2(n - 1) + c_4(n - 1) + c_5 \left(\frac{n(n + 1)}{2} - 1 \right) \\ &\quad + (c_6 + c_7) \left(\frac{n(n - 1)}{2} \right) + c_8(n - 1) \\ &= \frac{1}{2}(c_5 + c_6 + c_7)n^2 + (c_1 + c_2 + c_4 + \frac{1}{2}(c_5 - c_6 - c_7))n \\ &\quad - (c_2 + c_4 + c_5 + c_8) \Leftarrow \text{funzione quadratica} \end{aligned}$$

Nota: $\sum_{k=2}^n k = \frac{n(n+1)}{2} - 1$

Analisi degli algoritmi

Analisi del caso peggiore e del caso medio

- Solitamente si considera come complessità di un algoritmo il tempo di esecuzione con input che rappresenta **il caso peggiore**.
- Ragioni:
 - ① È una garanzia sulla descrizione del comportamento dell'algoritmo.
 - ② In alcuni casi, il caso peggiore è il più frequente.
 - ③ Il caso medio spesso è simile al caso peggiore (vedi INSERTIONSORT).

Analisi degli algoritmi

Tasso di crescita, ordine di grandezza

- Già considerando la RAM e INSERTIONSORT, si sono usate costanti c_k per rappresentare i costi delle operazioni (*costi astratti*).
- Queste costanti poi possono essere raggruppate perché danno un'informazione che non si usa: $(c_2 + c_4 + c_5 + c_8)$ può essere vista come una costante generica a .
- **Ultima astrazione:**
 - È più importante valutare la velocità con cui cresce il tempo di esecuzione, **tasso di crescita**, che i dettagli delle costanti.
 - Quindi, di un $T(n) = an^2 + bn + c$ è sufficiente considerare solo an^2 .
 - Visto che la costante a è astratta, la si assorbe considerando **l'ordine di grandezza**: $\Theta(n^2)$.
- Un algoritmo è considerato più efficiente di un altro se il suo tempo di computazione nel caso peggiore ha un tasso di crescita inferiore.

- INSERTIONSORT è stato progettato usando un metodo/approccio **incrementale**.
- Consideriamo ora un altro approccio che permette di risolvere lo stesso problema in modo più efficiente: **metodo divide et impera**
 - **Divide**: l'istanza viene spezzata in due o più parti più piccole;
 - **Impera**: ciascuna parte viene risolta ricorsivamente;
 - **Combina**: le soluzioni delle sottoparti vengono combinate per determinare la soluzione.

Progettare gli algoritmi

MERGESORT

MERGESORT è un algoritmo di ordinamento costruito con il metodo divide et impera.

- **Divide:** l'istanza di n elementi viene spezzata in 2 sottosequenza di $n/2$ elementi.
- **Impera:** ordina le due sottosequenze in modo ricorsivo (quando la sottosequenza ha dimensione 1, la ricorsione finisce).
- **Combina:** **fonde** le due sottosequenze ordinate.

L'operazione chiave è quella di **fondere** due sottosequenza ordinate.

Progettare gli algoritmi

MERGESORT

MERGE-SORT(A, p, r)

```
1  if  $p \geq r$                       // zero or one element?
2      return
3   $q = \lfloor (p+r)/2 \rfloor$               // midpoint of  $A[p:r]$ 
4  MERGE-SORT( $A, p, q$ )              // recursively sort  $A[p:q]$ 
5  MERGE-SORT( $A, q+1, r$ )            // recursively sort  $A[q+1:r]$ 
6  // Merge  $A[p:q]$  and  $A[q+1:r]$  into  $A[p:r]$ .
7  MERGE( $A, p, q, r$ )
```

- Dato $A[1:n]$, MERGESORT viene invocato con la chiamata MERGE-SORT($A, 1, n$).

MERGE(A, p, q, r)

```
1   $n_L = q - p + 1$                 // length of  $A[p:q]$ 
2   $n_R = r - q$                      // length of  $A[q+1:r]$ 
3  let  $L[0:n_L-1]$  and  $R[0:n_R-1]$  be new arrays
4  for  $i = 0$  to  $n_L - 1$              // copy  $A[p:q]$  into  $L[0:n_L-1]$ 
5       $L[i] = A[p+i]$ 
6  for  $j = 0$  to  $n_R - 1$              // copy  $A[q+1:r]$  into  $R[0:n_R-1]$ 
7       $R[j] = A[q+j+1]$ 
8   $i = 0$                            //  $i$  indexes the smallest remaining element in  $L$ 
9   $j = 0$                            //  $j$  indexes the smallest remaining element in  $R$ 
10  $k = p$                            //  $k$  indexes the location in  $A$  to fill
11 // As long as each of the arrays  $L$  and  $R$  contains an unmerged element,
12 //   copy the smallest unmerged element back into  $A[p:r]$ .
13 while  $i < n_L$  and  $j < n_R$ 
14     if  $L[i] \leq R[j]$ 
15          $A[k] = L[i]$ 
16          $i = i + 1$ 
17     else  $A[k] = R[j]$ 
18          $j = j + 1$ 
19          $k = k + 1$ 
20 // Having gone through one of  $L$  and  $R$  entirely, copy the
21 //   remainder of the other to the end of  $A[p:r]$ .
22 while  $i < n_L$ 
23      $A[k] = L[i]$ 
24      $i = i + 1$ 
25      $k = k + 1$ 
26 while  $j < n_R$ 
27      $A[k] = R[j]$ 
28      $j = j + 1$ 
29      $k = k + 1$ 
```

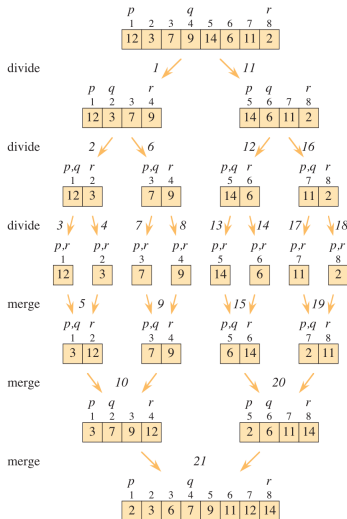
Progettare gli algoritmi

MERGESORT

Esempio di esecuzione di MERGESORT con $n = 8$ elementi

MERGE-SORT(A, p, r)

```
1  if  $p \geq r$                                 // zero or one element?
2      return
3   $q = \lfloor (p + r) / 2 \rfloor$                     // midpoint of  $A[p:r]$ 
4  MERGE-SORT( $A, p, q$ )                        // recursively sort  $A[p:q]$ 
5  MERGE-SORT( $A, q + 1, r$ )                    // recursively sort  $A[q + 1:r]$ 
6  // Merge  $A[p:q]$  and  $A[q + 1:r]$  into  $A[p:r]$ .
7  MERGE( $A, p, q, r$ )
```



Progettare gli algoritmi

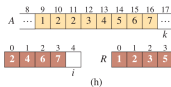
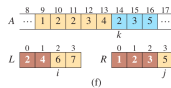
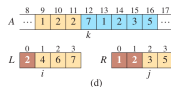
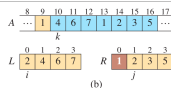
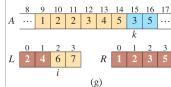
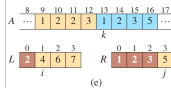
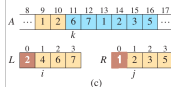
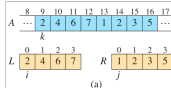
MERGESORT

MERGE(A, p, q, r)

```

1   $n_L = q - p + 1$       // length of  $A[p:q]$ 
2   $n_R = r - q$           // length of  $A[q+1:r]$ 
3  let  $L[0:n_L-1]$  and  $R[0:n_R-1]$  be new arrays
4  for  $i = 0$  to  $n_L - 1$  // copy  $A[p:q]$  into  $L[0:n_L-1]$ 
5       $L[i] = A[p+i]$ 
6  for  $j = 0$  to  $n_R - 1$  // copy  $A[q+1:r]$  into  $R[0:n_R-1]$ 
7       $R[j] = A[q+j+1]$ 
8   $i = 0$                 //  $i$  indexes the smallest remaining element in  $L$ 
9   $j = 0$                 //  $j$  indexes the smallest remaining element in  $R$ 
10  $k = p$                 //  $k$  indexes the location in  $A$  to fill
11 // As long as each of the arrays  $L$  and  $R$  contains an unmerged element,
12 //   copy the smallest unmerged element back into  $A[p:r]$ .
13 while  $i < n_L$  and  $j < n_R$ 
14     if  $L[i] \leq R[j]$ 
15          $A[k] = L[i]$ 
16          $i = i + 1$ 
17     else  $A[k] = R[j]$ 
18          $j = j + 1$ 
19      $k = k + 1$ 
20 // Having gone through one of  $L$  and  $R$  entirely, copy the
21 //   remainder of the other to the end of  $A[p:r]$ .
22 while  $i < n_L$ 
23      $A[k] = L[i]$ 
24      $i = i + 1$ 
25      $k = k + 1$ 
26 while  $j < n_R$ 
27      $A[k] = R[j]$ 
28      $j = j + 1$ 
29      $k = k + 1$ 

```



Progettare gli algoritmi

MERGESORT

MERGE(A, p, q, r)

```
1   $n_L = q - p + 1$       // length of  $A[p : q]$ 
2   $n_R = r - q$           // length of  $A[q + 1 : r]$ 
3  let  $L[0 : n_L - 1]$  and  $R[0 : n_R - 1]$  be new arrays
4  for  $i = 0$  to  $n_L - 1$  // copy  $A[p : q]$  into  $L[0 : n_L - 1]$ 
5       $L[i] = A[p + i]$ 
6  for  $j = 0$  to  $n_R - 1$  // copy  $A[q + 1 : r]$  into  $R[0 : n_R - 1]$ 
7       $R[j] = A[q + j + 1]$ 
8   $i = 0$                 //  $i$  indexes the smallest remaining element in  $L$ 
9   $j = 0$                 //  $j$  indexes the smallest remaining element in  $R$ 
10  $k = p$                 //  $k$  indexes the location in  $A$  to fill
11 // As long as each of the arrays  $L$  and  $R$  contains an unmerged element,
12 //   copy the smallest unmerged element back into  $A[p : r]$ .
13 while  $i < n_L$  and  $j < n_R$ 
14     if  $L[i] \leq R[j]$ 
15          $A[k] = L[i]$ 
16          $i = i + 1$ 
17     else  $A[k] = R[j]$ 
18          $j = j + 1$ 
19      $k = k + 1$ 
20 // Having gone through one of  $L$  and  $R$  entirely, copy the
21 //   remainder of the other to the end of  $A[p : r]$ .
22 while  $i < n_L$ 
23      $A[k] = L[i]$ 
24      $i = i + 1$ 
25      $k = k + 1$ 
26 while  $j < n_R$ 
27      $A[k] = R[j]$ 
28      $j = j + 1$ 
29      $k = k + 1$ 
```

Nota:

- Questa versione è la più semplice di MERGE.
- Nella versione III del Cormen e in rete si trovano versioni più compatte che usano la tecnica della **sentinella**.

Invariante di ciclo (Esercizio 2.3–3):
*All'inizio di ogni ciclo **while** (righe 12–18), $A[p : k - 1]$ contiene, ordinati, gli $i + j$ elementi più piccoli di $L[0 : n_L - 1]$ e $R[0 : n_R - 1]$. $L[i]$ e $R[j]$ sono i più piccoli elementi dei rispettivi array che non sono stati copiati.*

Progettare gli algoritmi

MERGESORT

```
MERGE( $A, p, q, r$ )
1   $n_L = q - p + 1$       // length of  $A[p : q]$ 
2   $n_R = r - q$           // length of  $A[q + 1 : r]$ 
3  let  $L[0 : n_L - 1]$  and  $R[0 : n_R - 1]$  be new arrays
4  for  $i = 0$  to  $n_L - 1$  // copy  $A[p : q]$  into  $L[0 : n_L - 1]$ 
5       $L[i] = A[p + i]$ 
6  for  $j = 0$  to  $n_R - 1$  // copy  $A[q + 1 : r]$  into  $R[0 : n_R - 1]$ 
7       $R[j] = A[q + j + 1]$ 
8   $i = 0$                 //  $i$  indexes the smallest remaining element in  $L$ 
9   $j = 0$                 //  $j$  indexes the smallest remaining element in  $R$ 
10  $k = p$                 //  $k$  indexes the location in  $A$  to fill
11 // As long as each of the arrays  $L$  and  $R$  contains an unmerged element,
12 //   copy the smallest unmerged element back into  $A[p : r]$ .
13 while  $i < n_L$  and  $j < n_R$ 
14     if  $L[i] \leq R[j]$ 
15          $A[k] = L[i]$ 
16          $i = i + 1$ 
17     else  $A[k] = R[j]$ 
18          $j = j + 1$ 
19      $k = k + 1$ 
20 // Having gone through one of  $L$  and  $R$  entirely, copy the
21 //   remainder of the other to the end of  $A[p : r]$ .
22 while  $i < n_L$ 
23      $A[k] = L[i]$ 
24      $i = i + 1$ 
25      $k = k + 1$ 
26 while  $j < n_R$ 
27      $A[k] = R[j]$ 
28      $j = j + 1$ 
29      $k = k + 1$ 
```

- Invariante di ciclo: dimostrazione
- **Inizializzazione:** $k = p$.
 $A[p : k - 1]$ è vuoto. $i = j = 0$.
 $L[0]$ e $R[0]$ sono gli elementi più piccoli non copiati. ✓
- **Conservazione:** Assumiamo che $L[i] \leq R[j]$. $A[p : k - 1]$ contiene $i + j$ elementi più piccoli. Si aggiunge $L[i]$. $A[p : k]$ contiene $i + j + 1$ elementi più piccoli. Si incrementano k e i . L'invariante è valida all'inizio dell'iterazione successiva. Caso $L[i] > R[j]$ è simile. ✓

Progettare gli algoritmi

MERGESORT

MERGE(A, p, q, r)

```
1   $n_L = q - p + 1$       // length of  $A[p : q]$ 
2   $n_R = r - q$           // length of  $A[q + 1 : r]$ 
3  let  $L[0 : n_L - 1]$  and  $R[0 : n_R - 1]$  be new arrays
4  for  $i = 0$  to  $n_L - 1$  // copy  $A[p : q]$  into  $L[0 : n_L - 1]$ 
5       $L[i] = A[p + i]$ 
6  for  $j = 0$  to  $n_R - 1$  // copy  $A[q + 1 : r]$  into  $R[0 : n_R - 1]$ 
7       $R[j] = A[q + j + 1]$ 
8   $i = 0$                 //  $i$  indexes the smallest remaining element in  $L$ 
9   $j = 0$                 //  $j$  indexes the smallest remaining element in  $R$ 
10  $k = p$                 //  $k$  indexes the location in  $A$  to fill
11 // As long as each of the arrays  $L$  and  $R$  contains an unmerged element,
12 //   copy the smallest unmerged element back into  $A[p : r]$ .
13 while  $i < n_L$  and  $j < n_R$ 
14     if  $L[i] \leq R[j]$ 
15          $A[k] = L[i]$ 
16          $i = i + 1$ 
17     else  $A[k] = R[j]$ 
18          $j = j + 1$ 
19      $k = k + 1$ 
20 // Having gone through one of  $L$  and  $R$  entirely, copy the
21 //   remainder of the other to the end of  $A[p : r]$ .
22 while  $i < n_L$ 
23      $A[k] = L[i]$ 
24      $i = i + 1$ 
25      $k = k + 1$ 
26 while  $j < n_R$ 
27      $A[k] = R[j]$ 
28      $j = j + 1$ 
29      $k = k + 1$ 
```

- **Conclusione:** Ciascuna iterazione incrementa o i o j . Alla fine o $i \geq n_L$ (quindi tutti gli elementi di L sono stati copiati) o $j \geq n_R$ (quindi tutti gli elementi di R sono stati copiati).
- Per l'invariante di ciclo, al termine $A[p : k - 1]$ contiene $i + j$ elementi più piccoli, ordinati, di $L[0 : n_L - 1]$ e di $R[0 : n_R - 1]$.
- A seconda che $i \geq n_L$ o $j \geq n_R$ uno dei cicli **while** (righe 20–23) o **while** (righe 24–27) viene eseguito per copiare i rimanenti elementi nella parte finale dell'array.

Progettare gli algoritmi

MERGESORT

MERGE(A, p, q, r)

```
1   $n_L = q - p + 1$       // length of  $A[p : q]$ 
2   $n_R = r - q$           // length of  $A[q + 1 : r]$ 
3  let  $L[0 : n_L - 1]$  and  $R[0 : n_R - 1]$  be new arrays
4  for  $i = 0$  to  $n_L - 1$  // copy  $A[p : q]$  into  $L[0 : n_L - 1]$ 
5       $L[i] = A[p + i]$ 
6  for  $j = 0$  to  $n_R - 1$  // copy  $A[q + 1 : r]$  into  $R[0 : n_R - 1]$ 
7       $R[j] = A[q + j + 1]$ 
8   $i = 0$                 //  $i$  indexes the smallest remaining element in  $L$ 
9   $j = 0$                 //  $j$  indexes the smallest remaining element in  $R$ 
10  $k = p$                 //  $k$  indexes the location in  $A$  to fill
11 // As long as each of the arrays  $L$  and  $R$  contains an unmerged element,
12 //   copy the smallest unmerged element back into  $A[p : r]$ .
13 while  $i < n_L$  and  $j < n_R$ 
14     if  $L[i] \leq R[j]$ 
15          $A[k] = L[i]$ 
16          $i = i + 1$ 
17     else  $A[k] = R[j]$ 
18          $j = j + 1$ 
19      $k = k + 1$ 
20 // Having gone through one of  $L$  and  $R$  entirely, copy the
21 //   remainder of the other to the end of  $A[p : r]$ .
22 while  $i < n_L$ 
23      $A[k] = L[i]$ 
24      $i = i + 1$ 
25      $k = k + 1$ 
26 while  $j < n_R$ 
27      $A[k] = R[j]$ 
28      $j = j + 1$ 
29      $k = k + 1$ 
```

Complessità:

- Sia $n = r - p + 1$
- Per costruire L e R :
 $\Theta(n_L + n_R) = \Theta(n)$
- Il ciclo **while** (righe 12–18) termina quando uno dei due array si esaurisce, quindi richiede al più $n_L + n_R - 1 = n - 1$ iterazioni: $O(n)$
- Solo uno dei due cicli successivi viene eseguito; nel complesso, la copia degli elementi rimanenti richiede al più n iterazioni: $O(n)$
- Totale: $\Theta(n)$

Progettare gli algoritmi

Equazione di ricorrenza (cap. 4)

- Algoritmo ricorsivo $\Rightarrow$ il tempo di computazione può essere descritto da **un'equazione di ricorrenza**
- Se si generano a sottoproblemi di dimensione n/b ciascuno e per spezzare il problema serve tempo $D(n)$ e poi per fondere le soluzioni tempo $C(n)$, il tempo totale è dato dalla soluzione dell'equazione di ricorrenza:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n \leq c \\ aT(n/b) + D(n) + C(n) & \text{altrimenti} \end{cases}$$

Progettare gli algoritmi

Analisi MERGESORT

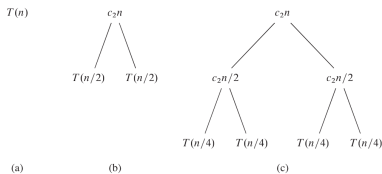
- Inizialmente si assume $n = 2^l$
- Si generano $a = 2$ sotto problemi di dimensione $n/2$ ciascuno; per spezzare il problema serve tempo $D(n) = \Theta(1)$; per fondere le soluzioni tempo $C(n) = \Theta(n)$

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ 2T(n/2) + \Theta(n) & n > 1 \end{cases}$$

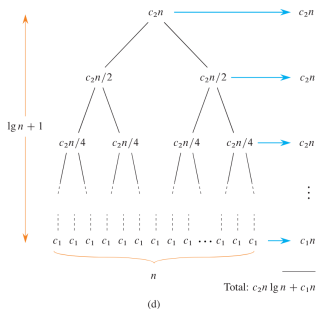
- Per il teorema dell'esperto: $T(n) = \Theta(n \lg n)$, dove $\lg = \log_2$.

Progettare gli algoritmi

Analisi MERGESORT



- Metodo grafico per risolvere un'equazione di ricorrenza: **albero di ricorsione**
- Numero di livelli dell'albero: $\lfloor \lg n \rfloor + 1$



In questa lezione si è visto:

- Due modi diversi per progettare un algoritmo che risolve il problema dell'ordinamento
- Il modello RAM per definire il costo astratto in tempo delle istruzioni
- Il tasso di crescita del tempo di computazione
- Primi esempi di calcolo del tempo di computazione (complessità). Equazioni di ricorrenza

- Studiare cap. 2 [CLRS].
- Risolvere gli esercizi 2.1–2, 2.1–4, 2.2–2, 2.2–4, 2.3–6 (solo iterativa), 2.3–7, 2.3–8 [CLRS].
- Risolvere il problema 2–4 [CLRS].